

# **Bring Your Own Project (BYOP) Report**

**Project: A\* Pathfinding Algorithm (Graph-Based)**

**Course: Fundamentals of AI/ML – VIT Bhopal, Semester 2**

**Author: Shashwat Mishra (25BCE10515)**

**Date: 28/03/2026**

---

## **1. Introduction**

Artificial Intelligence (AI) enables machines to solve problems that traditionally require human intelligence. One core concept is **search algorithms**, which allow systems to find optimal paths or solutions efficiently.

This project implements the **A\* search algorithm**, a heuristic-based graph search technique, to find the shortest path between a start node and a goal node in a weighted graph.

---

## **2. Objective**

- Demonstrate the **A\* search algorithm** in Python.
  - Solve shortest path problems in graphs using **heuristic-based optimization**.
  - Apply AI concepts learned in the course in a practical problem.
- 

## **3. Problem Statement**

Finding the shortest path is a common problem in navigation systems, robotics, and games. Traditional methods like BFS or DFS may explore unnecessary nodes, reducing efficiency. **A\*** improves efficiency by using a **heuristic function** to estimate distance to the goal, combining it with the actual cost to make informed decisions.

---

## **4. Tools and Technologies**

- **Python 3** – Programming language
  - **heapq module** – Implements a priority queue for A\*
  - **Text editor / IDE** – VS Code, PyCharm, etc.
- 

## **5. Methodology**

1. **Graph Representation** – Each node stores neighbors with edge costs.
2. **Heuristic Function** – Estimated cost ( $h(n)$ ) to reach goal.
3. **A\* Algorithm Logic:**

- Initialize **open list** (priority queue) with start node.
- Maintain  **$g(n)$**  (cost from start) and  **$f(n) = g(n) + h(n)$** .
- Expand the node with **lowest  $f(n)$** .
- Update paths for neighbours if a better cost is found.
- Repeat until goal is reached.

4. **Path Reconstruction** – Trace back from goal node using parent pointers.

---

## 6. Graph and Heuristic Example

**Graph:**

**Node Neighbours**

A    B(1), C(3)  
 B    D(1), E(5)  
 C    F(2)  
 D    G(1)  
 E    G(2)  
 F    G(1)  
 G    -

**Heuristic Values:**

**Node  $h(n)$**

A    7  
 B    6  
 C    4  
 D    3  
 E    2  
 F    1  
 G    0

**Start Node: A**

**Goal Node: G**

---

## 7. Sample Output

Shortest Path: **A** → **B** → **D** → **G**

---

## 8. Applications

- Navigation systems (e.g., Google Maps)
  - Game AI for pathfinding
  - Robotics and automated systems
  - Network routing and optimization
- 

## 9. Limitations

- Requires a well-defined heuristic for efficiency.
  - Performance depends on graph size and heuristic accuracy.
  - Not suitable for dynamic graphs without modifications.
- 

## 10. Future Enhancements

- Implement grid-based visualizations (Tkinter or Pygame).
  - Allow user input for dynamic graphs.
  - Compare with BFS and DFS performance.
  - Integrate with real-world map APIs.
- 

## 11. Conclusion

This project demonstrates the **practical application of the A\* search algorithm**, reinforcing AI/ML fundamentals such as heuristic functions, priority-based search, and path optimization. It lays the foundation for more complex AI projects in navigation, robotics, and intelligent systems.

---

## 12. References

1. Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach*.
2. Python heapq documentation: <https://docs.python.org/3/library/heapq.html>
3. A\* Algorithm Tutorials and Examples